

Building a 24-Bit Ultrasonic Acoustic-Emission DAQ System

A complete guide from component selection through field deployment

System at a glance

- 24-bit resolution, 500 kSPS per channel, dual-channel simultaneous sampling.
- Usable bandwidth 5 Hz to 116 kHz after digital compensation.
- 103.2 dB SNR, 16.9 ENOB, about 19.9 μ V RMS noise floor.
- Field-portable and battery-powered
- Built from commercial Analog Devices and Digilent evaluation hardware.

This guide is a document companion to the project repository. It walks through the full build: what to gather, how to wire it, how to bring it up, how to calibrate it, and how to run it in the field. Each major stage links to a deeper companion readme file in the repo.

Contents

1. System Overview	2
2. Before Starting	4
3. Component List.....	4
4. Hardware Assembly	5
5. Software Installation	7
6. Bring-Up Verification	8
7. Calibration.....	9
8. Data Acquisition Workflow	9
9. Digital Compensation	11
10. Field Deployment.....	12
11. Reference	13

1. System Overview

This system captures impact-excited acoustic emissions from a metallic test article, which the documentation calls a specimen. A short mechanical impact, such as a hammer strike, makes the specimen ring. The ring-down carries information about the material and its condition. The job of the DAQ is to record that faint, fast signal cleanly enough to analyze it later.

1.1 What the system does

An acoustic-emission event is small and brief. It lives in the kilohertz range and decays in milliseconds. To capture it well, the front end has to be quiet, the converter has to be both fast and high-resolution, and the data path has to stay clean from the sensor all the way to the laptop. This build pairs a broadband ultrasonic sensor with a 24-bit converter running at 500 kSPS on each of two channels, then moves the data off an embedded Linux board over isolated Gigabit Ethernet.

The result is an instrument that records two synchronized channels at microvolt-level resolution, runs for a full day on a battery, and fits in a portable case.

1.2 Architecture

The signal chain runs from a broadband ultrasonic acoustic-emission (UAE) sensor, into the AD4630-24's on-board analog front end, to the converter, then off the ZedBoard to a host laptop over galvanically isolated Gigabit Ethernet:

```
UAE sensor + preamp -> ADA4945-1 FDA (AFE, on the eval board) -> AD4630-24 ADC  
-> ZedBoard (Zynq-7020, Linux + IIO) -> Gigabit Ethernet (isolated) -> laptop
```

Three building blocks carry the whole system, and each one is described below.

The UAE sensor

The UAE sensor captures impact-excited emissions across the band of interest and presents a balanced (differential) output. That balanced output matches the converter's input range directly, so the chain needs no signal conditioning beyond the front end already on the eval board. The sensor runs from its own isolated low-voltage supply, kept separate from the digital electronics for noise reasons covered in section 1.3.

The ADC eval board

The EVAL-AD4630-24FMCZ carries the AD4630-24 converter and its analog front end. The converter provides 24-bit resolution at up to 2 MSPS. Here it runs at 500 kSPS per channel, dual-channel simultaneous, which gives a 250 kHz Nyquist limit on each channel. The board mates to the ZedBoard through the FMC-LPC connector and draws its power through that same connector, so it needs no separate supply cable.

The ZedBoard

The Digilent ZedBoard hosts a Xilinx Zynq-7020, which combines FPGA fabric with a dual-core ARM Cortex-A9 running embedded Linux. Analog Devices publishes pre-built HDL designs, Linux drivers, and Python libraries for this exact converter, so the build needs no custom FPGA development. The ADC shows up to Linux as a standard IIO device named ad4630-24, and a small set of iio command-line tools handle capture.

1.3 Design decisions

A few choices shape the rest of the build. Each one trades convenience for measurement quality, and each is worth understanding before assembly.

Gigabit Ethernet, not USB. I chose Ethernet for the data link because the RJ-45 magnetics give galvanic isolation for free. That isolation breaks the ground loop that would otherwise inject mains hum and switching noise into a microvolt-level measurement. USB ties the host and instrument grounds together and would need separate isolation hardware to reach the same noise floor.

LiFePO4 battery power. A LiFePO4 pack holds 12 to 13 V across more than 90% of its discharge. A typical three-cell Li-ion pack sags toward 9 V as it empties, which can make the system unstable partway through a session. The flat discharge curve keeps the rails steady from a full charge down to nearly empty.

A separate, isolated analog supply. I run the sensor and preamp from an isolated low-noise 5 V rail, never from anything derived on the ZedBoard. The board's DDR3 memory, Ethernet PHY, and SoC switching all put noise on its local rails, and any of that would couple straight into the signal if the preamp shared those rails.

Capture, then transfer. Each capture is buffered locally on the ZedBoard first, then transferred to the laptop afterward. A network hiccup during a capture cannot drop samples, because the samples are already safe in board memory before the transfer starts.

No external anti-alias filter. This is a deliberate change from the early plan. The eval board's own front end, the ADA4945-1 feedback network, already rolls off from roughly 48 to 55 kHz, far below the 250 kHz Nyquist at 500 kSPS, and the source is inherently band-limited. A dedicated analog anti-alias stage in front of the converter would change nothing measurable. I designed and built a 4th-order Butterworth filter anyway and characterized it, but it lives in the repo as a spare rather than in the signal chain. Its parts list is in section 3.4, and its full schematic and measured response are in repo docs/08.

1.4 Performance specifications

Parameter	Specification	Notes
ADC resolution	24-bit	True resolution; 16.9 ENOB measured
Sample rate	500 kSPS/ch (to 2 MSPS)	250 kHz Nyquist
Channels	2, simultaneous	Dual-sensor or differential pair
Usable bandwidth	5 Hz to 116 kHz	Post-compensation; raw AFE -3 dB about 48 to 55 kHz
SNR	103.2 dB	10 M-sample capture
Noise floor	about 19.9 μ V RMS	Shorted input
Sample-rate stability	± 40 ppm	
Input range	± 5 V differential	Matches preamp output
Data interface	Gigabit Ethernet	about 50 to 80 MB/s practical
Buffer	512 MB DDR3	about 128 s at 500 kSPS
Battery runtime	10+ hours	153 Wh LiFePO4 pack

Reading the headline numbers

ENOB (effective number of bits) is the resolution the system actually delivers once noise is included. A 24-bit converter never reaches 24 clean bits in practice. The measured 16.9 ENOB corresponds to the 103.2 dB SNR and is an honest figure for a working instrument.

Usable bandwidth needs a short explanation. The front end is already well into its rolloff above about 50 kHz. That rolloff is measured and then corrected in post-processing (see section 9 and repo docs/07), which restores usable response out to roughly 116 kHz before a digital regularization wall takes over. The raw analog response and the compensated response are two different things, and both are listed so the numbers are not misread.

2. Before Starting

A short orientation before the parts arrive. This section sets expectations on skill level, tools, time, and the handling habits that keep a sensitive instrument safe.

2.1 What this build assumes

This is an integration project rather than a board-bring-up project. It assumes basic comfort with a Linux command line, a serial console, and static IP networking. It does not assume any FPGA or RTL work, because the FPGA image, drivers, and capture tools are all pre-built by Analog Devices. No custom PCBs are required for the deployed system. The optional anti-alias filter in docs/08 is the only soldered board, and it is not needed to get the system running.

2.2 Tools and supplies

- A laptop running Windows, Linux, or macOS, with a free Ethernet port or a known-good USB-to-Ethernet adapter.
- A USB-A to Micro-B cable for the serial console.
- A terminal program that speaks 115200 8N1 over a serial port.
- A multimeter for checking supply voltages and jumper continuity before power-up.
- An anti-static wrist strap and mat. The converter front end is sensitive, and a static discharge can degrade it silently.

2.3 Time budget

Plan for two to four weeks end to end. Most of that is parts lead time. The hands-on assembly and software install take a few days of focused work. A detailed phase-by-phase timeline is in section 11.4.

2.4 Handling and safety

- Always set jumpers and verify them before applying power. A wrong VADJ setting can damage the ADC board the instant power is applied.
- Connect and verify every signal cable before connecting the 12 V supply. Power goes on last.
- Treat the eval board as ESD-sensitive at all times. Ground yourself before touching it.
- Keep the analog supply isolated from the digital supply, as described throughout this guide.

3. Component List

Every part is a standard distributor item. The lists below give the component and the quantity. Specific suppliers and ordering are left to preference.

3.1 Core system

Component	Qty
UAE sensor and preamp (broadband ultrasonic AE, balanced output, isolated supply input)	1
AD4630-24 ADC evaluation board (EVAL-AD4630-24FMCZ)	1
Digilent ZedBoard (Zynq-7020)	1
Kuiper Linux SD card (preconfigured, ships with the ADC board)	1

Choose a sensor whose differential output sits within the ± 5 V converter input range when using a 5 V reference, and which takes an isolated low-voltage supply input. That combination lets the sensor drive the front end directly, with no extra conditioning.

3.2 Power

Component	Qty
LiFePO4 battery pack, 12 V (about 150 Wh for a full day of runtime)	1
DC barrel adapter, 5.5 x 2.1 mm to 5.5 x 2.5 mm	1
Isolated low-noise 5 V supply (LDO module or quality power bank)	1

The ZedBoard uses a 5.5 by 2.5 mm barrel jack. Most packs output 5.5 by 2.1 mm, which is why the adapter is on the list.

3.3 Cables and adapters

Component	Qty
Cat6 Ethernet cable (about 2 m)	1
SMA to SMA cable (RG316, about 1 m)	1
BNC-to-SMA adapter (female to male)	1
USB-A to Micro-B cable (UART console)	1

3.4 Optional spare anti-alias filter

These parts build the 4th-order Butterworth Sallen-Key anti-alias filter that was designed and characterized as a spare. It is not part of the deployed chain (see section 1.3). Its schematic and measured response are in repo docs/08.

Component	Qty
Low-noise op-amp (ADA4896-2 or OPA2210)	1
10 kohm 0.1% precision resistor	4
220 pF C0G capacitor	2
330 pF C0G capacitor	2
100 pF C0G capacitor	2
680 pF C0G capacitor	2

4. Hardware Assembly

Companion document: repo docs/03 (ZedBoard and ADC setup).

Assembly is mechanical and quick once the jumpers are right. The one rule that matters most: verify every jumper and every connection before any power reaches the boards. Incorrect jumper settings can damage components, so check each setting against the tables below before applying power.

4.1 ZedBoard jumpers

The boot-mode jumpers tell the Zynq to load from the SD card. Set JP7 through JP11 as follows:

Jumper	Position
JP7	GND
JP8	GND
JP9	3V3
JP10	3V3
JP11	GND

VADJ select (J18): set it to 2.5 V. This sets the FMC I/O bank voltage, and the EVAL-AD4630-24FMCZ requires 2.5 V I/O. The ZedBoard ships with J18 at 1.8 V, so this jumper must be changed before the eval board is installed. A wrong VADJ setting can damage the ADC board the instant power is applied, so confirm it before anything else. Short JP6 as well, which the ZedBoard requires for SD-card boot. JP2 and JP3 are USB-OTG Vbus jumpers for the J13 OTG port and are not needed here; the J14 UART console used during setup is self-powered over its own USB cable and needs no jumper.

4.2 Mechanical sequence

1. Insert the Kuiper Linux SD card into the J12 slot on the underside of the ZedBoard.
2. With power disconnected, align the eval board's FMC connector with the ZedBoard's J1 FMC-LPC connector and press until fully seated. The connector should engage smoothly without forcing. The ADC board draws its power through the FMC connector, so there is no separate power cable.
3. Do not connect 12 V until all signal connections are made and verified.

4.3 Signal connections

1. Connect the UAE sensor output to the BNC-to-SMA adapter.
2. Run the SMA cable from the adapter to the eval board's IN0+. The ADA4945-1 front end accepts either a single-ended or a differential source. For a single-ended source, terminate the unused input (IN0-) at the equivalent source impedance instead of tying it straight to ground. For a differential source, drive both legs. The ADA4945-1 inverts the signal it sends to the ADC, so a positive input arrives on IN0-. This polarity flip does not matter for the AC ring-down bursts measured here, but keep the input wiring consistent with the wiring used during calibration.
3. Connect Ethernet from ZedBoard J10 directly to the laptop, with no switch in between.
4. Connect USB from the laptop to ZedBoard J14 (UART) for the serial console during setup. This connection is only needed during configuration and can be removed for field use.

4.4 Power connections

ZedBoard: route the LiFePO4 12 V through the DC5521-to-DC5525 adapter to barrel jack J20. The jack accepts 12 V at up to 3 A, and typical draw is around 10 to 15 W including the ADC board.

Preamp: power it from a separate isolated 5 V supply, and do not power it from ZedBoard USB. The board's 5 V rails carry DDR3, Ethernet-PHY, and SoC switching noise that would couple straight into the signal. The preamp draws about 500 mA, so a small isolated LDO module or a clean power bank is enough.

4.5 Grounding

- Ground cable shields at one end only, preferably the ADC end. Grounding both ends creates a ground loop.
- The Ethernet magnetics already provide 1500 to 2500 Vrms of isolation, which is the reason Ethernet is preferred over USB for the data link.
- Keep analog cables physically separated from digital and power runs.

5. Software Installation

Companion document: [repo docs/03](#).

There is no image to build and no driver to compile. The SD card ships with Kuiper Linux, the FPGA image, and the IIO stack already in place. Setup is mostly networking and a couple of installs on the host.

5.1 ZedBoard first boot

1. Apply 12 V to J20. Wait about 30 s. The red LD0 LED blinks once Linux is up.
2. Open a serial console at 115200 8N1 on the USB-UART (J14). Log in with the Kuiper default `analog / analog`.
3. Set a static IP in `/etc/network/interfaces`:

```
auto eth0
iface eth0 inet static
    address 192.168.1.100
    netmask 255.255.255.0
    gateway 192.168.1.1
```

Then restart networking with `sudo systemctl restart networking`.

4. Confirm the IIO daemon with `systemctl status iiod`. Expect active (running). It listens on port 30431, which the bench tests in section 6 use to reach the board over network IIO.

5.2 Host PC install

Install libiio on the host. The commands differ by operating system.

Windows

Download the libiio MSI installer from the Analog Devices libiio releases page, run it, then verify the install in a Command Prompt with `iio_info -V`.

Linux (Ubuntu or Debian)

```
sudo apt-get update
sudo apt-get install libiio-utils libiio-dev python3-pip
pip3 install pyadi-iio pylibiio numpy scipy matplotlib h5py
```

macOS

```
brew install libiio
pip3 install pyadi-iio pylibiio numpy scipy matplotlib h5py
```

With libiio installed, set the host Ethernet adapter to the same subnet (for example 192.168.1.10, netmask 255.255.255.0) and verify the link reaches the board:

```
iio_info -u ip:192.168.1.100      # should enumerate ad4630-24 + sampling_frequency
```

5.3 MATLAB (optional)

For direct capture from MATLAB using the Analog Devices Precision Toolbox, you need:

- MATLAB R2021b or later.
- Communications Toolbox, DSP System Toolbox, and Signal Processing Toolbox.
- Analog Devices Precision Toolbox, which provides the `adi.AD4630_24.Rx` object.

Install the Precision Toolbox from the MATLAB Add-On Explorer or the Analog Devices wiki.

MATLAB is optional. The deployed workflow runs from Python, and the analysis can run in either.

6. Bring-Up Verification

Before trusting a capture in the field, confirm the basics on the bench. These checks take a few minutes and catch the most common wiring and configuration mistakes.

An interactive UART diagnostic and recovery guide is available in [docs/zedboard-uart-diagnostics.html](https://docs.zedboard-uart-diagnostics.html). It covers board-side network checks, IIO verification, finite local capture testing, clean shutdown, and recovery when Ethernet access is unavailable.

6.1 Link and device check

With the board booted and the static IP set, enumerate the device from the host:

```
iio_info -u ip:192.168.1.100
```

The output should show the `ad4630-24` device with two voltage channels, a `sampling_frequency` attribute, and buffer capability. If the device is not found, verify network reachability with a ping, and confirm `iiod` is running on the ZedBoard. If `iio_info` connects but the device is missing, the FPGA bitstream did not load: power-cycle, reseal the SD card, and re-check that `VADJ` is 2.5 V.

6.2 Basic capture test

Pull a tiny buffer to confirm the data path end to end:

```
iio_readdev -u ip:192.168.1.100 -b 1000 ad4630-24 > test.bin  
ls -la test.bin      # should show a non-zero file size
```

6.3 Python acquisition test

This short script captures a buffer through `pyadi-iio` and plots it, which confirms the full host toolchain:

```
import adi  
import matplotlib.pyplot as plt  
  
adc = adi.ad4630(uri='ip:192.168.1.100', device_name='ad4630-24')  
adc.rx_buffer_size = 4096  
adc.sample_rate = 500000  
data = adc.rx()  
print(f'Captured {len(data[0])} samples from {len(data)} channels')  
  
plt.figure()  
plt.plot(data[0][:1000])
```



```
plt.title('AD4630-24 first capture')
plt.xlabel('Sample')
plt.ylabel('ADC code')
plt.show()
```

If this runs and shows captured data, the host side is fully operational.

6.4 Noise-floor check

With the input shorted, a clean system reads near its specified floor of about $19.9 \mu\text{V}$ RMS. A reading well above that points to a grounding or supply problem. The usual cause is the preamp drawing from a noisy rail instead of the isolated 5 V supply, or a cable shield grounded at both ends.

6.5 First real capture

Apply a light impact to the specimen with the sensor in place and take a short capture. A good capture shows a clear ring-down that rises sharply and decays smoothly, sitting well above the noise floor. If the signal clips, reduce the impact energy or move the sensor. If it barely registers, check the sensor supply and the signal-path connections.

7. Calibration

Companion document: [repo docs/05](#); script `scripts/calibrate_dual.py`.

Calibration turns raw converter counts into volts, one channel at a time, and gives a direct linearity check as a side benefit.

Raw ADC counts convert to volts through a per-channel linear fit, $V = \text{GAIN} \times \text{count} + \text{OFFSET}$. Rather than solving from just two points, the calibration steps through a spread of DC levels, for example -4 V to +4 V, and fits each channel by least-squares regression. That approach reports an R-squared and a residual sigma, which together are a direct linearity check, and by driving both channels to the same level it also exposes any offset mismatch between them.

Representative constants:

Channel	GAIN (V/count)	OFFSET (V)
Ch0	0.0000006268	-0.040402
Ch1	0.0000006355	+0.000751

These constants travel with every capture's metadata, so the count-to-volt conversion is fully reproducible downstream from the raw data alone.

8. Data Acquisition Workflow

Companion document: [repo docs/04](#); script `scripts/start_daq_uae.py`.

The deployed workflow favors reproducibility over convenience. It buffers each capture on the board, pulls it back, and saves only the raw data plus the metadata needed to reconstruct everything else.

8.1 Capture architecture

For manually triggered burst captures of about 10 s, buffering to DDR and then transferring is strongly preferred over continuous streaming. It guarantees sample continuity and keeps the acquisition code simple. The data rates are modest, and the math behind them is worth seeing:

Parameter	Value	How it is computed
Sample rate	500 kSPS	selected for the band of interest
Channels	2	differential pair or dual sensors
Bits per sample	24 (stored in 32)	packed in a 32-bit container
Data rate	about 4 MB/s	500k x 2 ch x 4 bytes
10 s capture	about 40 MB	fits easily in 512 MB DDR3
Transfer time	about 0.8 s	at 50 MB/s Ethernet

A 10 s capture is about 40 MB, roughly 8% of the 512 MB DDR3, so buffer pressure is never a concern at these lengths. The capture file lands in /tmp, whose backing store (RAM or the SD card) depends on the installed Kuiper image, so check the mount with `findmnt /tmp` and `df -h /tmp` before assuming RAM-speed storage. The choice between buffering and streaming comes down to this:

Approach	Advantages	Trade-offs
Capture to DDR, then transfer	guaranteed continuity, simpler code, full bandwidth	about 0.8 s latency after the capture
Continuous streaming	unlimited duration, live monitoring	network jitter risk, more complex sync

For manual burst capture, where you start recording, allow a few strikes, then stop, the capture-then-transfer approach removes any risk of dropped samples from network timing.

8.2 The deployed capture (start_daq_uae.py)

The capture runs locally on the ZedBoard and is pulled back afterward. The two commands that do the work on the board are:

```
iio_attr -u local: -d ad4630-24 sampling_frequency 500000 && \
iio_readdev -u local: -b 200000 -s <N> ad4630-24 > /tmp/capture.bin
```

The host script connects over SSH (paramiko), starts the capture, prompts the operator to apply the impact excitation, pulls the file back over SFTP, and parses it. Parsing reshapes the stream into two interleaved channels and shifts each sample to recover the signed 24-bit result (`reshape(-1,2)` then `>> 8`). It then calibrates, applies the Method B compensation for a quick on-screen quality glance, and saves.

Save format, raw counts only. The .mat file stores the raw int32 counts plus full metadata: the calibration constants, every compensation parameter, the system-ID arrays, and the sample rate. The calibrated and compensated signals are not saved. They are recomputed downstream from the raw counts plus metadata. Saving raw keeps the files small, prevents derived data from drifting out of sync with the method, and lets any old capture be re-derived if the processing later improves.

8.3 Higher-level capture methods

For quick tests, two higher-level options exist on top of the same IIO interface. They are convenient, but note that the deployed workflow in section 8.2 is what saves raw counts plus metadata. Treat these as bench tools.

MATLAB Precision Toolbox

```
rx = adi.AD4630_24.Rx;
rx.uri = 'ip:192.168.1.100';
rx.SamplesPerFrame = 500000; % 10 s at 500 kSPS
```

```
rx.EnabledChannels = [1, 2];

disp('Recording, apply the impact excitation now');
data = rx();
disp('Capture complete');
release(rx);
```

Python (pyadi-iio)

```
import adi
import numpy as np
from scipy.io import savemat
import time

adc = adi.ad4630(uri='ip:192.168.1.100', device_name='ad4630-24')
adc.rx_buffer_size = 5000000 # 10 s at 500 kSPS
adc.sample_rate = 500000

print('Recording, apply the impact excitation now')
data = adc.rx()
print('Capture complete')

savemat('capture.mat', {
    'ch0': data[0],
    'ch1': data[1],
    'sample_rate': adc.sample_rate,
    'timestamp': time.time(),
})
print(f'Saved {len(data[0])} samples to capture.mat')
```

Calling Python from MATLAB

MATLAB can also drive pyadi-iio directly through its Python engine, which is handy if the rest of your analysis already lives in MATLAB:

```
pyenv('Version', '/usr/bin/python3'); % adjust path as needed
adi = py.importlib.import_module('adi');

adc = adi.ad4630(uri='ip:192.168.1.100', device_name='ad4630-24');
adc.rx_buffer_size = int32(5000000);
adc.sample_rate = int32(500000);

py_data = adc.rx();
ch0 = double(py.numpy.array(py_data{1}));
ch1 = double(py.numpy.array(py_data{2}));
```

8.4 Handing off to the analysis

A saved capture loads straight into the analysis. A quick look at one channel, in the time domain and as a spectrum:

```
load('capture.mat')
t = (0:length(ch0)-1) / sample_rate;

figure
subplot(2,1,1)
plot(t*1000, ch0)
xlabel('Time (ms)'); ylabel('ADC code'); title('Channel 0, raw capture')

subplot(2,1,2)
[pxx, f] = pwelch(double(ch0), [], [], [], sample_rate);
plot(f/1000, 10*log10(pxx))
```

```
xlabel('Frequency (kHz)'); ylabel('PSD (dB/Hz)'); title('Power spectral density')
```

From there, the repo's burst-quality analysis (scripts/daq_burst_quality.m, with companion docs/10) detects and scores each strike, then saves the results with metadata: specimen ID, measurement point, and timestamp.

9. Digital Compensation

Companion document: repo docs/07.

The front end rolls off inside the band of interest, so its response is measured once and corrected in software. The same compensator runs everywhere the data is touched, which keeps results consistent.

Because the front end rolls off within the band of interest (section 1.3), the response is characterized and then corrected in post-processing. The identical compensator runs in the capture quick-look, in the noise baseline, and in the analysis pipeline, so a captured ring-down looks the same no matter which tool opens it.

The method, called Method B, is a non-parametric Wiener deconvolution with minimum-phase reconstruction. The front-end magnitude is measured at 24 swept-sine frequencies and interpolated in the log domain. Its phase is reconstructed from the magnitude through the minimum-phase (cepstral/Hilbert) relationship. The response is then inverted as $G(f) = \text{conj}(H) / (|H|^2 + \text{epsilon}(f))$. The regularizer $\text{epsilon}(f)$ is small in-band and ramps up through a soft wall near 135 kHz, which stops the inverse from amplifying out-of-band noise.

One point is worth being precise about. That digital wall near 135 kHz is not anti-aliasing. Only the analog front end ahead of the converter limits the band before sampling. The wall is a numerical safeguard on the inverse filter, nothing more.

10. Field Deployment

Companion document: repo docs/09.

In the field the priorities are a reliable link, a steady analog supply, and a repeatable strike-and-capture loop. The checklists below keep a session from stalling on something avoidable.

10.1 Pre-deployment checklist

- Fully charge the LiFePO4 battery (about 10 h at 15 W draw).
- Verify the isolated 5 V preamp supply.
- Confirm the host reaches the ZedBoard (iio_info).
- Run one full capture end-to-end in the lab.
- Pack spare cables (Ethernet, SMA, BNC adapter).
- Confirm the laptop has storage for the capture files.

10.2 Field setup

1. Position the case near the test point. Keep the laptop stable, and shaded if possible.
2. Power up: LiFePO4 to ZedBoard J20 via the adapter, and the isolated 5 V to the preamp.
3. Connect the signal: sensor to BNC-SMA adapter to eval board IN0+.
4. Connect Ethernet: ZedBoard J10 directly to the laptop.

5. After about 30 s of boot, set the laptop to a static IP on the same subnet, for example 192.168.1.10, and verify with a ping or `iio_info`.

10.3 Data collection loop

Repeat this loop at each measurement point:

1. Place the sensor at the designated point on or near the specimen.
2. Start the capture and let it begin buffering.
3. Apply the impact excitation (hammer strikes) while recording.
4. The capture transfers automatically once the buffer fills, or on an early stop, and the quick-look comes up.
5. Run the burst-quality analysis (repo docs/10) to detect and score each strike, then save with metadata: specimen ID, measurement point, and timestamp.

10.4 Troubleshooting

No network. Check that both ends are on the same subnet first. Reseat the Ethernet at both ends, then swap the cable to rule out field damage.

ad4630-24 not found. The bitstream did not load. Power-cycle, reseat the SD card, and check that VADJ is 2.5 V.

Noisy captures. Confirm the preamp is on the isolated 5 V supply. Check that shields are grounded at one end only, and that analog cables are away from digital and power runs.

Capture timeout or missing samples. Lower the sample rate or buffer size to confirm the basics work, and clear any competing network traffic. The deployed capture pulls the file over SSH and SFTP, so make sure port 22 is reachable. Port 30431 only matters for the network-IIO bench captures in sections 6 and 8.3.

When the network is wedged, the UART console still works with no Ethernet at all, which makes it the fallback path for diagnosing the board.

11. Reference

11.1 Component summary

Component	Qty
UAE sensor and preamp	1
AD4630-24 ADC evaluation board (EVAL-AD4630-24FMCZ)	1
Digilent ZedBoard (Zynq-7020)	1
Kuiper Linux SD card	1
LiFePO4 battery pack, 12 V	1
DC barrel adapter	1
Isolated 5 V supply	1
Cat6 Ethernet cable	1
SMA to SMA cable	1

Component	Qty
BNC-to-SMA adapter	1
USB-A to Micro-B cable	1
Optional spare anti-alias filter parts (see section 3.4)	1 set

11.2 Network settings

Parameter	Value
ZedBoard IP	192.168.1.100 (in /etc/network/interfaces)
Subnet mask	255.255.255.0
IIO daemon port	30431 (network IIO, bench tests)
SSH / SFTP (deployed capture)	22 (capture pull rides SSH)
Host PC IP	192.168.1.x (not .100)
Connection	direct Ethernet, no switch

11.3 Software resources

Resource	Location
Analog Devices HDL	github.com/analogdevicesinc/hdl
pyadi-iio	github.com/analogdevicesinc/pyadi-iio
libiio releases	github.com/analogdevicesinc/libiio/releases
Kuiper Linux	wiki.analog.com (Kuiper Linux)
MATLAB Precision Toolbox	wiki.analog.com (Precision Toolbox)
AD4630-24 datasheet	analog.com/AD4630-24
ZedBoard docs	digilent.com (ZedBoard)